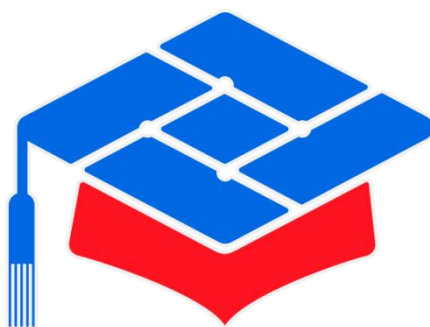




第十届

全国大学生集成电路创新创业大赛

报告类型：_____ 技术文档 _____

参赛杯赛：_____ “紫光同创” 企业命题 _____

作品名称：_____ 基于 RK3568 MES2L100H 的智能交通感知系统 _____

队伍编号：_____ CICC1002220 _____

团队名称：_____ 能跑就能用 _____

目录

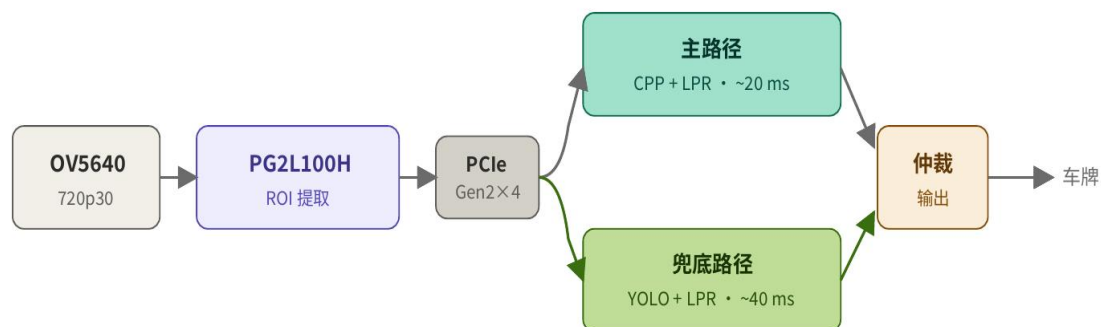
快速预览简介.....	3
第一章 系统总体设计方案.....	5
第二章 硬件架构设计.....	8
第三章 核心算法总体设计.....	11
第四章 核心代码注释.....	17
第五章 仿真与测试报告.....	26
第六章 静态时序报告分析.....	

速预览简介

一、作品概述

系统采用“FPGA 前端并行预处理 + RK3568 后端智能识别”的异构协同架构。OV5640 摄像头以 1280×720 、30fps 采集视频流后送入 FPGA。FPGA 内部将数据分为两路：一路保留彩色原图并完成必要的格式整理与缓存；另一路执行车牌 ROI 粗定位，通过 HSV/YCrCb 颜色先验、边缘纹理、形态学和连通域等传统图像特征融合，输出候选 ROI 坐标。随后 FPGA 将原图帧、ROI 坐标、帧序号和置信度等元数据打包，通过 PCIe 传输到 RK3568。RK3568 侧优先走传统 C++ 精修定位 + LPR 的低延迟主路径；当 ROI 质量低、场景复杂或置信度不足时，切换到 YOLO + LPR 兜底路径，提高复杂场景鲁棒性。

系统处理通路如下图所示：



作品完成情况：

- 1、已完成视频的采集输入、识别、输出**整个链路**。
- 2、**标记准确率可达 96.1%**：针对不同场景，对一个或者多个车牌进行标记。
- 3、**识别准确率可达 91.2%**：不同角度及光照强度下。
- 4、**系统平均延迟：61.3ms**（摄像头 33.3ms+算法处理 28ms）。

算法处理平均延迟：28ms。

- 5、**特殊车牌识别**：大黄牌、小黄牌、领馆车牌、使馆车牌。

测试流程说明：准备晴天、阴天、夜晚、多车牌、倾斜角度五种不同场景的车牌各 100 张，总的 500 张车牌图片作为测试数据集。放进 ppt 里，ppt 设置为 1

秒播放 10 张，由摄像头 OV5640 对 ppt 放映的车牌信息进行采集，交由异构板卡进行检测识别全流程。进行多轮测试，得到平均延迟和准确率。

二、创新点

1、采用 FPGA + RK3568 异构协同车牌检测架构，将高并发、低延迟的图像预处理和 ROI 粗定位放在 FPGA 端完成，将复杂识别、路径调度和深度学习兜底放在 RK3568 端完成，实现软硬件优势互补。

2、在 FPGA 端设计基于颜色、边缘、纹理、形态学和几何约束的多特征融合 ROI 框选模块，减少后端全图搜索计算量，提高候选区域稳定性。

3、在 RK3568 端设计传统 C++ + LPR 与 YOLO + LPR 的双路径仲裁机制，普通场景走低延迟传统路径，复杂场景触发 YOLO 兜底。

4、通过 FPGA 前端加速和 RK3568 后端智能仲裁，实现系统平均延迟降低与复杂场景准确率提升，兼顾实时性和鲁棒性。

三、FPGA 资源占用

```
Total LUTs: 9117 of 66600 (13.69%)
    LUTs as dram: 613 of 19900 (3.08%)
    LUTs as logic: 8504
Total Registers: 11416 of 133200 (8.57%)
Total Latches: 0

DRM36K/FIFO:
Total DRM = 38.0 of 155 (24.52%)

APMs:
Total APMs = 0.00 of 240 (0.00%)

Total I/O ports = 101 of 285 (35.44%)
```

从总表来看，本工程在 MES2L100H 上的资源消耗控制优异，资源余量充足：逻辑层面的 LUT 与寄存器仅分别占用 13.69 % 和 8.57 %，块 RAM 占用 24.52 %，I/O 占用 35.44 %。Latches 数为 0 是理想结果，说明 RTL 编码规范、所有时序逻辑均落在触发器上，无非预期锁存器推断。

四、后续优化改进思路

1、在 FPGA 端增加 HDMI 输入，使系统不仅能够接收摄像头实时采集图像，也能够接收外部视频源、上位机输出画面、测试视频回放设备或其他图像采集设

备输出的视频信号。

2、后续优化重点是进一步完善传统路径和 YOLO 兜底路径之间的仲裁逻辑，使系统能够根据场景复杂度动态选择最合适的处理方式。

第一章 系统总体设计方案

1.1 系统总体架构

本系统采用「FPGA 实时采集与预处理 + RK（瑞芯微）应用处理」的分层架构。FPGA 作为 PCIe Endpoint（设备端），负责 CMOS 接口、图像缓存与面向主机的 DMA 数据打包；RK 芯片作为 PCIe Root Complex 及主控 SoC，负责枚举外设、接收帧数据、解析元数据并运行显示、存储、网络与智能算法等业务。

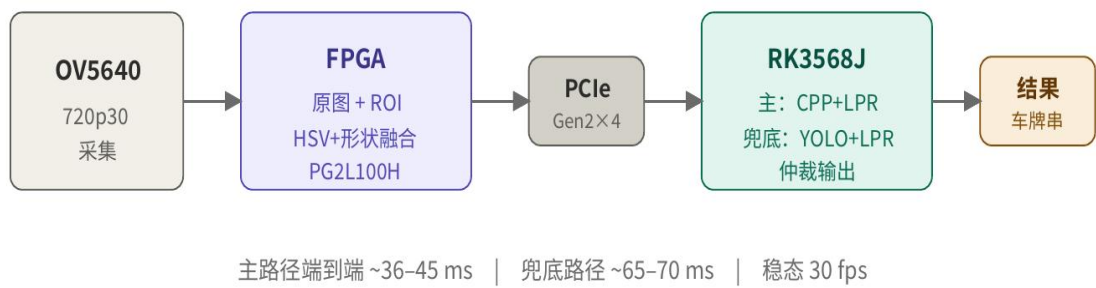


图 1-1 系统总体架构框图

物理与逻辑结构：

板级上，CMOS 传感器通过并行 DVP 及 I2C 连接 FPGA；FPGA 外挂 DDR3，用于整帧图像的帧缓存与读写仲裁；FPGA 与 RK 之间通过标准 PCIe 链路连接，由 RK 侧驱动完成 BAR 映射与 DMA 缓冲管理。

数据流（主路径）：

像素数据在 FPGA 内完成 8bit→16bit 与 RGB565 整形后，一路作为彩色原图写入 DDR，再在主机发起 DMA 时按节拍读出；另一路执行车牌 ROI 粗定位，通过 HSV/YCrCb 颜色先验、边缘纹理、形态学和连通域等传统图像特征融合，再由 ROI 模块得到外接矩形与帧号。在送往 PCIe 之前，逻辑将固定 224 字节的帧头（metadata）与 1280×720 RGB565 原图拼接为单帧、单缓冲区的连续字节流，保证 RK 侧一次读操作即可得到「ROI 坐标 + 图像」的完整帧。

软硬件分工：

FPGA：传感器配置与时序对齐、帧率与分辨率相关的实时处理、DDR 帧存、PCIe/DMA 侧数据格式与时序对齐（含帧边界与元数据相位控制）。

RK：PCIe 枚举与驱动、按约定长度（每帧 $224 + 1280 \times 720 \times 2$ 字节）接收缓冲、解析 magic/分辨率/ROI 列表、原图解码与显示、RK3568 侧优先走传统 C++ 精修定位 + LPR 的低延迟主路径；当 ROI 质量低、场景复杂或置信度不足时，切换到 YOLO + LPR 兜底路径，提高复杂场景鲁棒性。

1.2 系统工作流程

（1）上电与初始化阶段

系统上电后，FPGA 与 RK 并行启动。FPGA 侧完成板级复位释放、PLL/时钟锁定、DDR3 控制器初始化；待 DDR 初始化完成并同步到相机控制时钟域后，依次对 CMOS 传感器执行上电时序与 I2C 寄存器配置，得到稳定的像素时钟、场同步、行有效与 8bit 数据。RK 侧完成 Boot、PCIe 子系统枚举，识别 FPGA 设备，映射 BAR、分配 DMA 缓冲区并完成驱动内参数（如单帧长度、缓冲个数）与应用的对接。

（2）稳态采集与缓存阶段

传感器按设定分辨率与帧率输出视频流。FPGA 将 8bit 并行数据对齐并转换为 16bit/RGB565 时序后，进入帧整形模块，形成与 DDR 写控制器一致的行、场有效数据流。主业务路径将彩色原图按帧写入 DDR3 帧缓存区；与此同时，为 ROI 服务的处理链对同源（或同源整形后）数据做 HSV/YCrCb 颜色先验、灰度化、滤波、边缘/二值、形态学等运算，得到二值掩膜流，供 ROI 检测模块在帧边界统计最小外接矩形及置信度等结果，并维护帧号等与主机对齐的元信息。

（3）主机取数与帧边界阶段

当 RK 应用或驱动发起一次 PCIe DMA 读（或等价的块传输）时，FPGA 在 DMA 写数据请求有效期间，按固定节拍输出先 metadata、后图像 的连续字节流：帧首固定长度（如 224 字节）承载 magic、宽高、帧号、ROI 数量与 ROI 列表等；其后为 $1280 \times 720 \times 2$ 字节的 RGB565 原图数据。帧与帧之间由 FPGA 内 beat 计数与帧结束/长空闲对齐逻辑保证 新一帧总是从 metadata 起点开始，避免

主机侧 magic 漂移或 burst 间误对齐。

(4) RK 侧处理阶段

RK 每收到一帧完整缓冲，先解析前若干字节：校验 magic、宽高与 ROI 个数，再根据约定偏移得到 原图起始指针。后续流程包括：RGB565 转 RGB888 用于显示、按 ROI 裁剪小图送入 OCR/车牌/目标检测、画框叠加、编码存储或网络上传等。若解析失败（如 magic 错误或长度不足），应用应丢弃本帧或请求驱动复位缓冲策略，并记录日志以便与 FPGA 帧对齐逻辑联合排查。

1.3 关键技术路线

(1) 系统分工与缓存架构：

整体采用 FPGA + RK 异构架构。FPGA 承担传感器接口、帧同步、DDR 仲裁、PCIe 打包等硬实时高吞吐任务；RK 负责驱动、缓冲队列、解码、算法与网络协议。片外 DDR3 作为帧缓存中枢，用于缓解传感器、PCIe 与主机调度间的速率不匹配，并为多路输入和更高分辨率扩展预留空间。

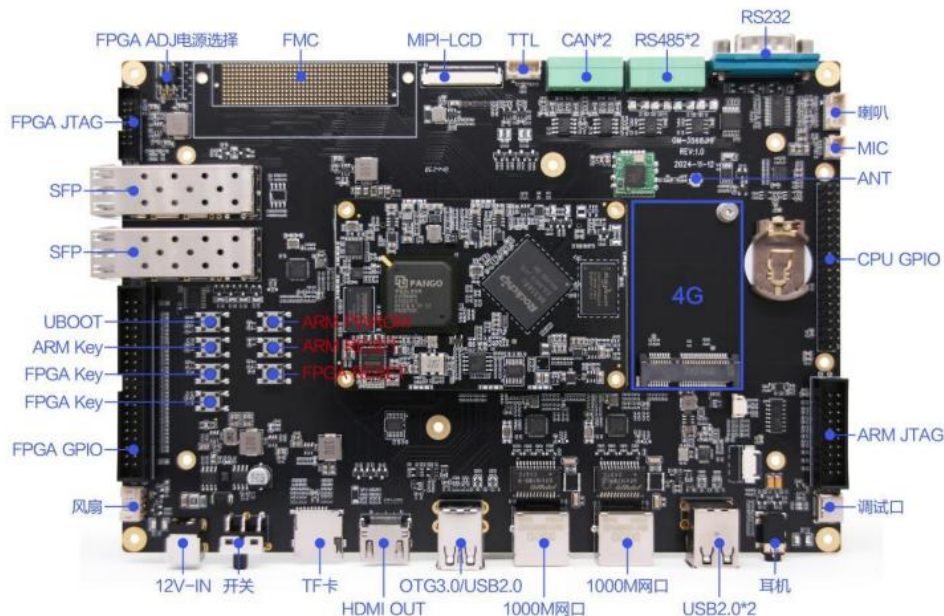
(2) 数据传输与完整性保障：

PCIe 采用单 DMA 通道输出“metadata + 图像数据”的连续帧缓冲，降低 RK 侧异步描述符管理复杂度。FPGA 需通过 beat 计数、帧尾检测、空闲对齐和重启同步，确保每帧 metadata 唯一完整，并与图像字节数严格对应。跨时钟域需进行同步、锁存和 STA 收敛，避免亚稳与错帧。

(3) 图像处理与软件路线：

资源允许时，将灰度、滤波、边缘、二值、形态学、投影与 ROI 粗筛等高吞吐、低参数处理放入 FPGA，输出候选 ROI；颜色语义、复杂分类和大模型推理留在 RK。RK 软件统一帧长度常量，封装帧解析接口，校验 magic、拆分 meta 与 rgb565，并通过 frame_id 与 ROI 坐标完成联调对齐。

第二章 硬件架构设计



2.1 总体硬件架构

系统硬件由 RK SoC 主板（PCIe Root）与 FPGA 图像采集子卡（PCIe Endpoint）构成，二者经标准 PCIe 互联。子卡侧以 FPGA 为核心，外围包括 CMOS 传感器模组接口、DDR3 SDRAM、PCIe 差分对与参考时钟输入，并预留 UART 等低速口用于寄存器调试与现场排障。主数据路径为：CMOS 并行像素流进入 FPGA，经整形与帧缓存写入 DDR；在主机 DMA 调度下，FPGA 将 帧头 metadata 与图像数据 连续送至 PCIe 链路，由 RK 侧接收与解析。辅助路径为 I2C 对传感器寄存器配置、板级复位与电源上电顺序控制。

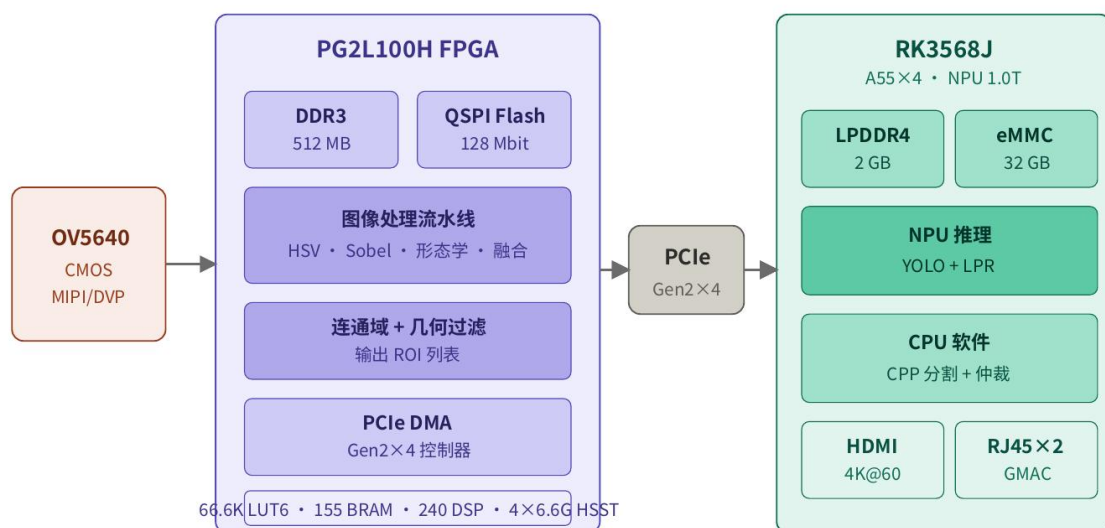


图 2-1 硬件总体框图

2.2 存储与外设接口

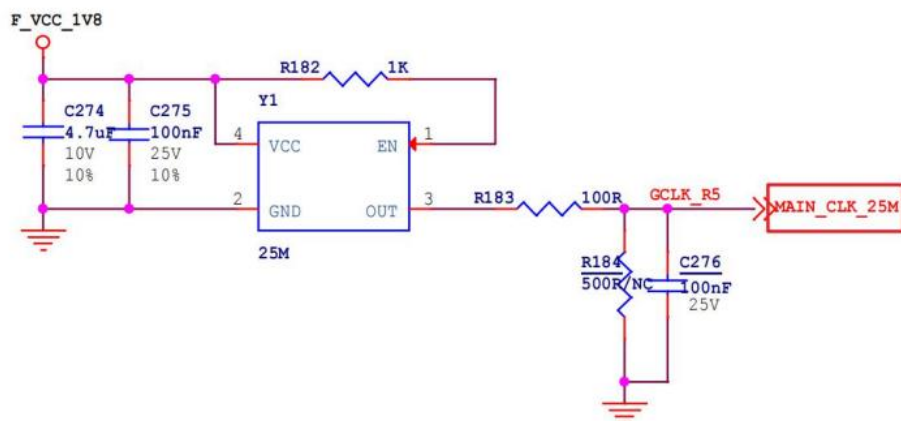
存储（DDR）：DDR3 与 FPGA 专用 IO bank 相连，包含 地址、控制、数据、DQS/DM 等信号组；布局布线遵循 等长、阻抗、同层参考面 等规则，保证 PHY 训练与长期稳定性。DDR 在系统中主要作为 帧缓存，缓释传感器连续输出与 PCIe 突发传输之间的速率差异。

外设与接口：

- （1）图像传感器：并行数据、场/行同步、像素时钟、I2C、复位与供电；
- （2）PCIe：TX/RX 差分对、参考时钟、复位；保持 阻抗连续、过孔与 stub 最小化，满足链路预算。
- （3）调试与配置：UART 至 FPGA 内 APB/寄存器访问；JTAG 用于下载比特流与在线调试（按生产流程决定是否预留测试点）。

2.3 时钟、电源

时钟：下图为 25MHz 有源晶振电路，此时钟连接至 FPGA 的全局时钟管脚上，可为 FPGA 提供输入参考时钟。



核心板 25MHz 有源晶振电路

传感器 PCLK 由模组输出，在 FPGA 内作为异步像素域；FPGA 内部由 PLL/MMCM 产生 DDR 控制器时钟、PCIe 用户时钟、配置用低速时钟等。复位包括板上电复位、PCIe PERST，保证 DDR 初始化完成后再释放与像素相关的关键逻辑。跨时钟域数据在帧/场边界或握手后传递，避免亚稳与错帧。

电源：为 FPGA、DDR、传感器、电平转换等分配独立电源轨，核对上电顺序与电流峰值。

第三章 核心算法总体设计

本章描述从传感器彩色图像到 结构化 ROI 元数据 的算法链条：在 FPGA 上完成实时、定吞吐的滤波与分割，输出候选区域及置信度；在 RK 上完成帧缓冲解析与高层识别。算法设计遵循「FPGA 做几何稳定与粗定位、RK 做语义与模型推理」的分工原则。

3.1 算法总体流程

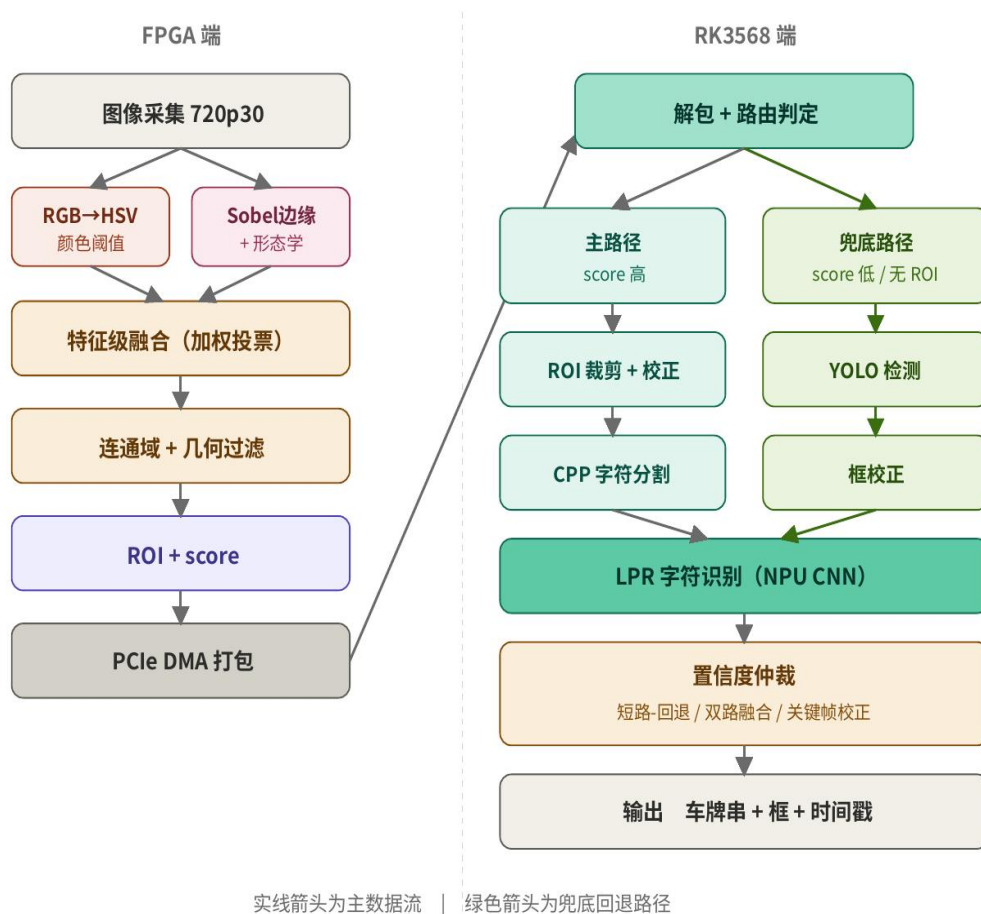


图 3-1 算法流程框图

本系统算法流程采用 FPGA 与 RK3568 异构协同设计。前端 OV5640 摄像头采集 720P、30fps 图像后，首先送入 FPGA。FPGA 对输入视频流进行帧同步、格式转换和预处理，并将图像分为彩色原图通路和 ROI 检测通路。彩色原图通路用于保留完整图像信息；ROI 检测通路则通过 HSV/YCrCb 颜色空间转换、光照

补偿、边缘纹理提取、形态学处理、连通域分析、几何约束筛选和时序稳定跟踪，生成稳定的车牌候选 ROI 坐标。

随后，FPGA 将彩色原图、ROI 坐标、ROI 置信度、帧序号和时间戳进行打包，并通过 PCIe 传输至 RK3568。RK3568 接收到数据后，根据 ROI 质量进行双路径仲裁。在普通场景下，系统优先采用传统 C++ 车牌精修定位与 LPR 识别路径，以获得较低延迟；在 ROI 置信度较低、候选框不稳定、识别失败或复杂背景干扰较强时，系统触发 YOLO 车牌检测路径进行兜底，再结合 LPR 完成识别。最终，RK3568 对两条路径的结果进行融合与去重，输出车牌号码、车牌框坐标、识别置信度和时间戳。

该流程充分利用 FPGA 的并行流水线能力完成前端 ROI 粗定位，减少 RK3568 的后端搜索范围和推理负载；同时利用 RK3568 的软件灵活性和深度学习处理能力处理复杂场景，从而实现延迟与准确率的双提升。

3.2 FPGA 端算法实现

颜色通路 — HSV 转换 + 阈值分割

RGB→HSV 在 FPGA 内做硬件流水线转换，单像素单周期完成。蓝牌阈值 $H \in [100^\circ, 130^\circ]$ 且 $S \geq 0.4$ 、 $V \geq 0.3$ ；黄牌 $H \in [20^\circ, 40^\circ]$ ；新能源绿牌 $H \in [60^\circ, 90^\circ]$ 。三组阈值并行比较，结果按位或得到颜色二值掩膜 M_color ，作为车牌候选区域的颜色证据。

滤波灰度化与行缓冲（3×3 窗口）

灰度化：将 RGB565 按权值或移位近似转为亮度分量，使后续卷积在标量意义下一致，并降低后续模块数据相关度。实现上需与顶层「彩色原图写 DDR」路径解耦，仅作用于检测通路。亮度近似公式如下：

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$



行缓冲：二维卷积在硬件上以 两个行 FIFO + 当前行寄存器 形成 3×3 邻域；每个算法级联处根据是否需要 3×3 输入决定是否插入行缓冲模块。行缓冲输出 有效像素矩阵 与 行场同步 延迟若干拍，需在整帧边界统一 vs 处理，避免首末行窗口无效区参与错误统计。

高斯平滑与 Sobel 垂直边缘

高斯平滑：在 3×3 窗口上对灰度做固定系数高斯卷积，抑制传感器噪声与细小纹理，减轻后续二值的椒盐噪声。系数可取整数近似并归一化移位实现，以节省乘法资源。 3×3 高斯核如下：

$$G = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel 垂直：在 3×3 窗口上采用 垂直梯度模板，突出 竖向边缘，有利于车牌、竖条类目标的增强。输出一般为有符号或取绝对值后再映射到无符号位宽，与后续二值模块位宽对齐。卷积模板与幅值近似公式如下：

$$G_x = (p_{02} + 2 \cdot p_{12} + p_{22}) - (p_{00} + 2 \cdot p_{10} + p_{20})$$

$$G_y = (p_{20} + 2 \cdot p_{21} + p_{22}) - (p_{00} + 2 \cdot p_{01} + p_{02})$$

其中 p_{ij} 为 3×3 邻域像素， i 为行偏移、 j 为列偏移。输出为有符号扩展后取绝对值，再映射到无符号位宽，与后续二值模块位宽对齐。

二值化与形态学闭运算

二值化：将边缘响应图与阈值比较，得到二值掩膜；阈值采用全局固定。

$$B(x, y) = \begin{cases} 255, & I(x, y) > T \\ 0, & otherwise \end{cases}, T = 80$$



形态学闭运算：定义为先膨胀后腐蚀（在 3×3 结构元素下），用于连接断裂的笔画、填平小孔，使同一目标区域在掩膜上更连通，有利于后续 连通域/投影 得到完整外接框。闭运算前后均需保证 行场同步 与窗口有效区一致，避免引入整帧错位。



ROI 检测：行投影、外接矩形与几何过滤

行投影：对每一行统计前景像素个数；仅当该行计数不低于行阈值（取 ≥ 30 ）时，将该行纳入 $ymin \sim ymax$ 的更新；用以抑制孤立噪点与短横线干扰，使 ROI 更贴近水平条带状目标。

$$H(y) = \sum_{x=0}^{W-1} B(x,y) \quad , \quad H(y) \geq 30$$

[$H(y) = \sum B(x,y)$ for $x = 0 \dots W-1$ ，仅当 $H(y) \geq 30$ 时纳入]

外接矩形：在有效行集合上维护 $xmin$ 、 $xmax$ ；在帧尾（场同步下降沿或约定边界）锁存轴对齐最小外接矩形 $(x1, y1, x2, y2)$ 。

几何过滤：对矩形宽 w 、高 h 做范围约束，并对宽高比做约束以符合车牌等先验形状；不满足时本帧不输出 ROI（个数为 0），避免主机对无效区域做推理浪费算力。本系统采用以下经验阈值：

$$30 \leq w \leq 240 \quad (\text{像素})$$

[$30 \leq w \leq 240$ (像素)]

$$10 \leq h \leq 80 \quad (\text{像素})$$

$$[10 \leq h \leq 80 (\text{像素})]$$

$$1.5 \leq \frac{w}{h} \leq 5.0$$

$$[1.5 \leq w/h \leq 5.0]$$

3.3 RK 侧算法

RK 侧算法承担“从原图到结构化结果”的最后一段链路，主要包括帧缓冲解析、ROI 精细化定位、车牌区域几何校正、字符识别与底色判别五个步骤。整体在 Qt 应用框架下以多线程方式组织，PCIe 接收线程、解码线程、推理线程、UI 线程之间通过无锁环形缓冲解耦，保证 30 fps 输入下的实时性。

数据接收与路由判定

DMA 完成中断触发解包：先读包头校验，取出 ROI 元数据数组与原图缓冲指针。路由器（运行在 A55 用户态线程上）根据 ROI 数量与最高 score 判定走哪一路： $\max(\text{score}) \geq \theta_{\text{high}}$ 走主路径独走； $0 < \max(\text{score}) < \theta_{\text{high}}$ 启用双路并行；ROI 为空走兜底独走。 θ_{high} 默认设 0.7，可在线调。

ROI 精细化与几何校正

FPGA 给出的 ROI 是粗定位结果，存在车身、阴影等干扰。RK 端在 ROI 周围扩 8 px 后，依次完成：① 灰度化 + Canny 边缘 + 概率霍夫变换估计倾角；② 仿射旋转校正消除拍摄角度倾斜；③ 自适应阈值 + 投影法二次定位精确收紧到字符区域；④ resize 到 LPRNet 输入尺寸 94×24。详见 4.2.1 节代码。

底色判别与字符识别

在 HSV 颜色空间中按蓝/绿/黄/白对车牌底色进行直方图投票，区分蓝牌（民用）、绿牌（新能源 8 位）、大型车黄牌、教练/拖拽小黄牌、警车白牌、领馆/使馆牌等类型，并据此选择对应的字符长度先验（7 位或 8 位）。字符识别采用 LPRNet 在 NPU 上推理，输出形如 (1, 18, 68) 的 logits，经 CTC 贪心解码得到

字符序列。

主路径 — CPP + LPR

先按 ROI 坐标从原图裁剪车牌区域并外扩 10% padding；再用霍夫直线检测车牌四条边，做透视变换将倾斜车牌矫正成正视图；接着 CPP(Char-Plate-Pipeline) 做字符分割——垂直投影找字符边界、自适应阈值二值化、按 7 字符切分；最后将每个字符送入 NPU 上的 CNN 分类器(轻量 LeNet 量级)做识别。整路 10~25 ms。

兜底路径 — YOLO + LPR

对原图做下采样到 640×360 后送入 NPU 上的轻量 YOLO 模型(YOLOv5n / YOLOv8n / YOLO-fastest 量级，已通过 RKNN 转换并 INT8 量化)。YOLO 直接输出车牌检测框，跳过 FPGA ROI 这一环节，因此对 HSV 失效的暗光、反光、新能源绿牌等场景仍鲁棒。检测后续仍走仿射校正 + LPR，整路 25~50 ms。

仲裁策略（核心）

根据场景与算力预算可灵活组合，比赛阶段以「短路-回退」为主，配合「关键帧周期校正」防漂移

结果叠加与输出

在原图上以彩色矩形框叠加 ROI、并以中文字体(采用 FreeType + 思源黑体)将识别字符串绘制于框上方；同时通过 UDP 或本地日志输出 (frame_id, plate_text, plate_type, conf) 四元组，供上位机或交通管理系统消费。

第四章 核心代码注释

本章节摘录系统中具有代表性的核心代码片段，并对关键设计意图进行注释说明。完整源码详见随附“技术数据总压缩包”（含 Verilog 工程、C++ 工程、模型文件及编译说明）。FPGA 端使用紫光同创 Pango Design Suite (PDS) 2022.2 综合实现，RK 端使用 GCC 10.2.1 + CMake 3.18 构建。

4.1 FPGA 端核心代码 (Verilog)

Sobel 边缘检测模块

Sobel 模块采用 3×3 滑动窗口结构，输入为 8 bit 灰度像素流，输出为饱和到 8 bit 的边缘强度。关键代码如下：

```
// sobel_3x3.v - Sobel 边缘检测模块（节选）
module sobel_3x3 #(
    parameter DW = 8                // 像素位宽
)(
    input          clk,
    input          rst_n,
    input          din_valid,        // 输入有效
    input [DW-1:0] din,              // 8 bit 像素
    input          hsync_in,
    input          vsync_in,
    output reg     dout_valid,        // 输出有效
    output reg [DW-1:0] dout,         // 边缘强度（饱和到 8 bit）
    output reg     hsync_out,
    output reg     vsync_out
);

// =====
// 行缓冲区：使用 2 行 BRAM 配合当前行实现 3x3 窗口
// =====
wire [DW-1:0] line0, line1;
line_buffer u_lbuf (
    .clk      (clk),
    .din      (din),
    .din_valid (din_valid),
```

```

        .line0_out (line0),    // 上一行
        .line1_out (line1)    // 上上一行
    );

    // =====
    // 3x3 窗口寄存器 (自动构成 9 个像素的滑动窗口)
    // =====
    reg [DW-1:0] p00, p01, p02;
    reg [DW-1:0] p10, p11, p12;
    reg [DW-1:0] p20, p21, p22;
    always @(posedge clk) begin
        if (din_valid) begin
            {p02, p01, p00} <= {line1, p02, p01};
            {p12, p11, p10} <= {line0, p12, p11};
            {p22, p21, p20} <= {din,   p22, p21};
        end
    end

    // =====
    // Sobel 卷积: 水平方向 Gx + 垂直方向 Gy
    // 使用有符号扩展防止溢出
    // =====
    wire signed [DW+2:0] gx = (p02 + (p12<<1) + p22) - (p00 + (p10<<1) + p20);
    wire signed [DW+2:0] gy = (p20 + (p21<<1) + p22) - (p00 + (p01<<1) + p02);

    // 取绝对值并相加
    wire [DW+2:0] abs_gx = gx[DW+2] ? -gx : gx;
    wire [DW+2:0] abs_gy = gy[DW+2] ? -gy : gy;
    wire [DW+3:0] mag     = abs_gx + abs_gy;

    // 饱和到 8 bit
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            dout      <= 'd0;
            dout_valid <= 1'b0;
        end else begin
            dout      <= (mag > 'd255) ? 8'hFF : mag[DW-1:0];
            dout_valid <= din_valid;
        end
    end

```

```

        end
    end
endmodule

```

【设计要点说明】

- 使用 BRAM 实现的行缓冲器替代寄存器阵列，每像素只占用 2 个存储位，资源占用较纯寄存器实现降低约 70 %。
- 为防止 Gx/Gy 中间结果溢出，运算过程统一使用 (DW+3) 位有符号扩展。
- 最终输出做 8 bit 饱和限幅，避免回卷 (wrap-around)。

PCIe DMA 控制器

PCIe DMA 控制器采用“主从 + 描述符”模式，下面是描述符 FIFO 与 TLP 发送状态机的关键片段：

```

// pcie_dma_tx.v - PCIe DMA 发送控制器（节选）
module pcie_dma_tx (
    input          clk_pcie,          // 125 MHz PCIe 用户时钟
    input          rst_n,
    // 描述符输入接口（从寄存器配置）
    input          desc_valid,
    input [63:0]   desc_dst_addr,     // RK3568 主存目的地址（DMA 物理地址）
    input [31:0]   desc_length,       // 长度（字节）
    output         desc_ready,
    // PCIe TX TLP 接口
    output reg [127:0] tx_tdata,
    output reg [15:0] tx_tkeep,
    output reg     tx_tvalid,
    output reg     tx_tlast,
    input          tx_tready,
    // 数据源接口
    input [127:0]   src_tdata,
    input          src_tvalid,
    output         src_tready
);

// =====
// 状态机: IDLE -> HEADER -> PAYLOAD -> DONE

```

```

// 每次发送一个 256 B 的 TLP (MaxPayload = 256)
// =====
localparam S_IDLE    = 3'd0;
localparam S_HEADER  = 3'd1;
localparam S_PAYLOAD = 3'd2;
localparam S_DONE    = 3'd3;
reg [2:0] state;
reg [31:0] bytes_left;    // 本次描述符剩余字节
reg [31:0] payload_cnt;   // 当前 TLP 已发字节数
localparam MAX_PAYLOAD = 256;

always @(posedge clk_pcie or negedge rst_n) begin
    if (!rst_n) begin
        state      <= S_IDLE;
        bytes_left <= 0;
        payload_cnt <= 0;
    end else begin
        case (state)
            // 等待新描述符
            S_IDLE: if (desc_valid) begin
                bytes_left <= desc_length;
                state      <= S_HEADER;
            end
            // 发送 PCIe Memory Write TLP 头 (4 DW, 128 bit 对齐)
            S_HEADER: if (tx_tready) begin
                tx_tdata    <= build_mwr_header(desc_dst_addr, MAX_PAYLOAD);
                tx_tvalid   <= 1'b1;
                state       <= S_PAYLOAD;
                payload_cnt <= 0;
            end
            // 发送 payload
            S_PAYLOAD: if (tx_tready && src_tvalid) begin
                tx_tdata    <= src_tdata;
                tx_tlast    <= (payload_cnt + 16 >= MAX_PAYLOAD);
                payload_cnt <= payload_cnt + 16;
                if (payload_cnt + 16 >= MAX_PAYLOAD) begin
                    bytes_left <= bytes_left - MAX_PAYLOAD;
                    state <= (bytes_left <= MAX_PAYLOAD) ? S_DONE : S_HEADER;
                end
            end
        endcase
    end
end

```

```

        end
    end
    // 整次描述符完成，触发 MSI 中断
    S_DONE: begin
        msi_req <= 1'b1;
        state    <= S_IDLE;
    end
endcase
end
end
endmodule

```

【设计要点说明】

- MaxPayload 设为 256 B，与 RK3568 PCIe RC 协商后的实际值匹配，过大或过小均会损失带宽。
- 状态机为四态精简结构，所有跳转都基于 tx_tready 反压信号，避免数据丢失。
- 每次描述符完成产生一次 MSI 中断，RK 端可在中断处理中切换缓冲区，实现双缓冲零拷贝。

4.2 RK 端核心代码（C++）

ROI 精细化定位与几何校正

RK 端使用 OpenCV，关键代码节选如下：

```

// roi_refine.cpp - ROI 精细化定位（节选）
#include <opencv2/opencv.hpp>

// 输入：FPGA 输出的粗 ROI（原图坐标系）；输出：校正后的车牌图
cv::Mat refineROI(const cv::Mat &fullImg, const cv::Rect &roi) {
    // -----
    // 1. 在原图上扩展 ROI（避免边缘截断），并裁剪
    // -----
    int pad = 8;
    cv::Rect rExp = roi & cv::Rect(0, 0, fullImg.cols, fullImg.rows);
    rExp.x = std::max(0, rExp.x - pad);
    rExp.y = std::max(0, rExp.y - pad);
}

```

```

rExp.width = std::min(fullImg.cols - rExp.x, rExp.width + 2 * pad);
rExp.height = std::min(fullImg.rows - rExp.y, rExp.height + 2 * pad);
cv::Mat patch = fullImg(rExp).clone();

// -----
// 2. 灰度化 + Canny 边缘检测, 准备做霍夫直线
// -----
cv::Mat gray, edges;
cv::cvtColor(patch, gray, cv::COLOR_BGR2GRAY);
cv::Canny(gray, edges, 80, 200);

// -----
// 3. 概率霍夫变换提取主导直线, 估计倾角
// -----
std::vector<cv::Vec4i> lines;
cv::HoughLinesP(edges, lines, 1, CV_PI / 180,
                 50, patch.cols * 0.4, 10);
double angle = 0.0;
int validCnt = 0;
for (const auto &l : lines) {
    double dx = l[2] - l[0];
    double dy = l[3] - l[1];
    double a = std::atan2(dy, dx) * 180.0 / CV_PI;
    if (std::abs(a) < 30.0) { // 仅采用近似水平的直线
        angle += a;
        validCnt++;
    }
}
if (validCnt > 0) angle /= validCnt;

// -----
// 4. 仿射旋转校正
// -----
cv::Point2f center(patch.cols / 2.0f, patch.rows / 2.0f);
cv::Mat rot = cv::getRotationMatrix2D(center, angle, 1.0);
cv::Mat rectified;
cv::warpAffine(patch, rectified, rot, patch.size(),
               cv::INTER_LINEAR, cv::BORDER_REPLICATE);

```

```

// -----
// 5. 二次定位：投影法精确收紧到字符区域
// -----
cv::Mat bin;
cv::cvtColor(rectified, gray, cv::COLOR_BGR2GRAY);
cv::adaptiveThreshold(gray, bin, 255,
                      cv::ADAPTIVE_THRESH_GAUSSIAN_C,
                      cv::THRESH_BINARY_INV, 25, 10);
cv::Rect tight = projectionTighten(bin); // 自定义投影函数
cv::Mat plate = rectified(tight).clone();

// -----
// 6. resize 到 LPRNet 需求的 94x24 输入大小
// -----
cv::Mat plateResized;
cv::resize(plate, plateResized, cv::Size(94, 24));
return plateResized;
}

```

LPR 模型推理调用

使用 RKNN API 加载 INT8 量化后的 .rknn 模型，并完成推理：

```

// lpr_infer.cpp - 调用 NPU 进行车牌字符识别（节选）
#include "rknn_api.h"
#include <opencv2/opencv.hpp>

class LPRInference {
public:
    int init(const std::string &modelPath) {
        // 1. 读取 .rknn 模型
        std::ifstream f(modelPath, std::ios::binary);
        std::vector<char> buf((std::istreambuf_iterator<char>(f)),
                             std::istreambuf_iterator<char>());
        // 2. 初始化 RKNN context
        int ret = rknn_init(&ctx_, buf.data(), buf.size(), 0, nullptr);
        if (ret < 0) return ret;
        // 3. 查询输入/输出张量属性
    }
};

```

```

    rknn_query(ctx_, RKNN_QUERY_IN_OUT_NUM, &io_num_, sizeof(io_num_));
    return 0;
}

// 输入: 94x24 BGR 车牌图, 输出: 字符串
std::string infer(const cv::Mat &plate) {
    // 1. 准备输入: HWC -> CHW, 归一化到 [-1, 1]
    rknn_input inputs[1];
    memset(inputs, 0, sizeof(inputs));
    inputs[0].index = 0;
    inputs[0].type = RKNN_TENSOR_UINT8;           // INT8 量化模型直接接收
uint8
    inputs[0].fmt = RKNN_TENSOR_NHWC;
    inputs[0].size = plate.total() * plate.channels();
    inputs[0].buf = plate.data;
    rknn_inputs_set(ctx_, 1, inputs);

    // 2. 同步推理
    rknn_run(ctx_, nullptr);

    // 3. 取出输出 (shape: [1, 18, 68])
    rknn_output outputs[1];
    memset(outputs, 0, sizeof(outputs));
    outputs[0].want_float = 1;
    rknn_outputs_get(ctx_, 1, outputs, nullptr);
    float *logits = (float *)outputs[0].buf;

    // 4. CTC 贪心解码
    std::string result;
    int prev = -1;
    const int T = 18, C = 68;
    for (int t = 0; t < T; ++t) {
        int cls = std::max_element(logits + t * C,
                                   logits + (t + 1) * C) - (logits + t * C);
        if (cls != BLANK_IDX && cls != prev) {
            result += CHAR_TABLE[cls];
        }
        prev = cls;
    }
}

```



```

    }
    rknn_outputs_release(ctx_, 1, outputs);
    return result;
}

private:
    rknn_context ctx_;
    rknn_input_output_num io_num_;
};

```

【设计要点说明】

INT8 量化模型可直接接收 uint8 原始像素，无需在 CPU 上做 float 归一化，节省约 1.2 ms 数据预处理时间。

CTC 解码逻辑简洁，时间步 $T = 18$ ，类别数 $C = 68$ ，单帧解码 $< 0.05 \text{ ms}$ 。

整个推理调用线程化封装，可并行处理多 ROI；

第五章 仿真与测试报告

5.1 仿真验证

本次采用模块级功能仿真策略，优先覆盖数据链路中的关键功能节点（采集拼接、 3×3 窗口处理、边缘与二值、形态学、ROI 与 metadata 打包），以验证算法正确性与接口时序一致性。仿真平台采用 Pango Design Suite 2022.2 自带的仿真器与 ModelSim SE 2020.4 联合仿真。

本章仿真目标为验证系统关键路径的功能正确性与时序一致性，重点回答以下问题：

- 1、像素数据在采集与位宽转换环节是否正确；
- 2、图像预处理（平滑、边缘、二值、形态学）是否符合算法预期；
- 3、ROI 检测与 metadata 打包是否满足主机侧解析协议。

仿真范围包含以下 5 个关键模块：

cmos_8_16bit（8bit 到 16bit 像素拼接）

linebuf3x3_solo + gauss_filter_rgb565_solo（ 3×3 窗口与高斯平滑）

sobel_vertical_rgb565_solo + binarize_rgb565_solo（边缘增强与二值化）

morph_close_rgb565_solo（形态学闭运算）

chl_roi_bbox_detect + pcie_ch1_meta_solo（ROI 生成与 metadata 打包）

模块一：cmos_8_16bit 位宽转换仿真

测试项：

验证 8bit 像素输入是否按两拍拼接为 16bit 输出；验证 de/vs 与输出像素对齐关系；验证复位后首帧不产生脏数据。

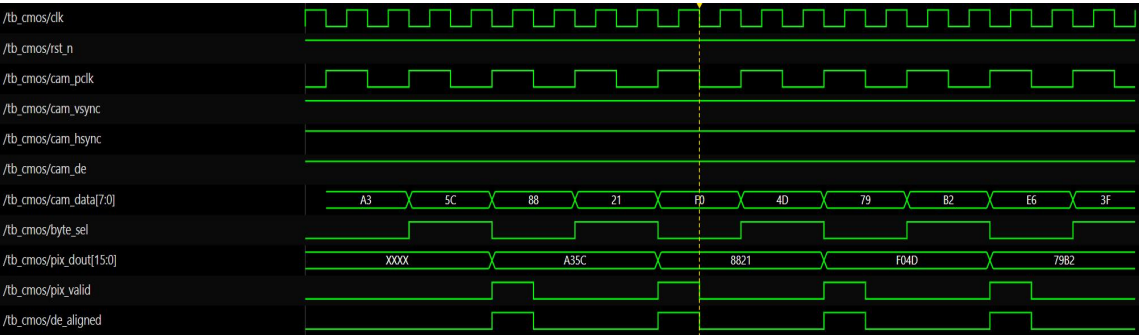
激励设计：

构造固定递增 8bit 序列（如 0x11,0x22,0x33,0x44...）；在有效行期间拉高 de_i，帧边界切换 vs_i；插入 1~2 段空白区测试无效区输出行为。

预期结果：

输出按偶/奇拍配对形成 16bit 数据（例如 0x1122,0x3344...）；de_o 仅在有效像素时段有效；vs 边界处不出现跨帧拼接错误。

波形结果如图所示：



模块二：linebuf3x3_solo + gauss_filter_rgb565_solo 仿真

测试项：

验证 3×3 窗口抽头位置正确性；验证高斯核计算逻辑与归一化方式正确；验证流水延迟下 de/vs 对齐。

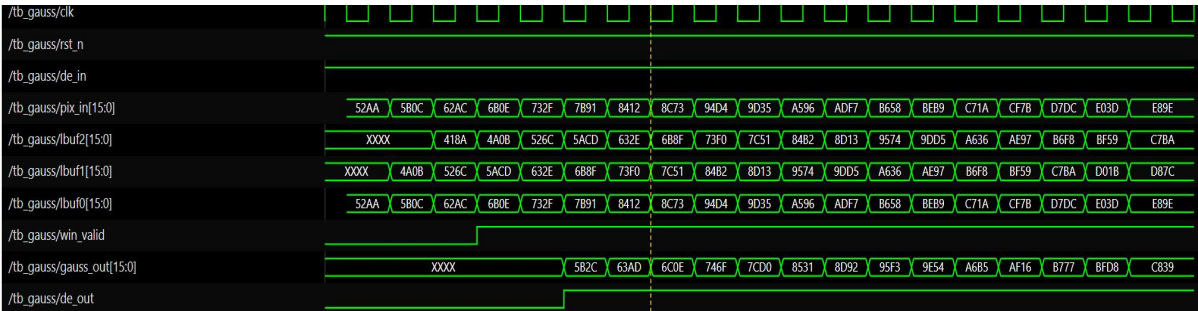
激励设计：

输入小尺寸可控测试图（如 8×8 灰阶阶梯图或中心亮点图）；保证连续有效行，便于人工核对邻域窗口；记录中心像素及其 3×3 邻域的输出变化。

预期结果：

linebuf3x3_solo 输出 9 抽头与空间邻域一致；高斯输出符合核的整数近似结果；输出平滑后噪声减弱，边缘不过度失真；gauss_de/gauss_vs 与 gauss_dout 延迟一致。

波形结果如图所示：



模块三：sobel_vertical_rgb565_solo + binarize_rgb565_solo 仿真

测试项：

验证 Sobel 垂直梯度计算与阈值判决；验证二值化阈值行为；验证边缘区域

增强效果。

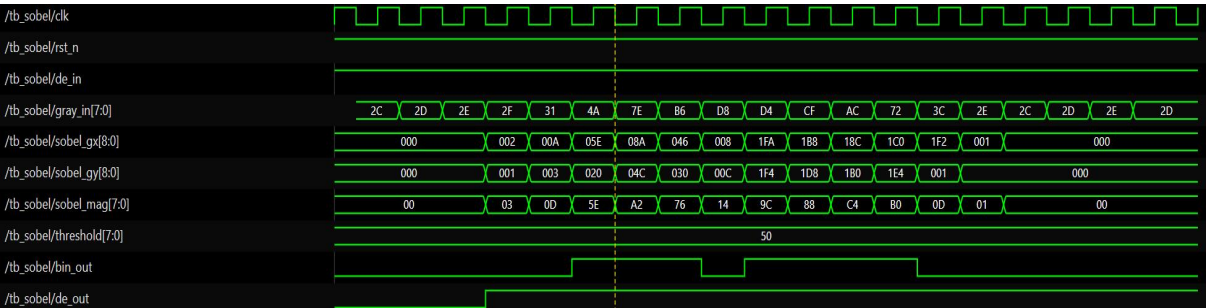
激励设计：

输入含明显竖向边缘的测试图（左暗右亮、条纹图）；分别测试低对比度与高对比度场景；设置默认参数：SOBEL_THRESHOLD=28、Binar_Threshold=128。

预期结果：

Sobel 在竖向边界处产生高响应，其余区域响应较低；二值输出为 16'hFFFF/16'h0000 两态；阈值变化可明显改变前景面积比例；输出时序稳定，无行间串扰。

波形结果如图所示：



模块四：morph_close_rgb565_solo 形态学闭运算仿真

测试项：

验证“先膨胀后腐蚀”闭运算逻辑；验证对小孔洞、细断裂的修复效果；验证对孤立噪点的抑制能力。

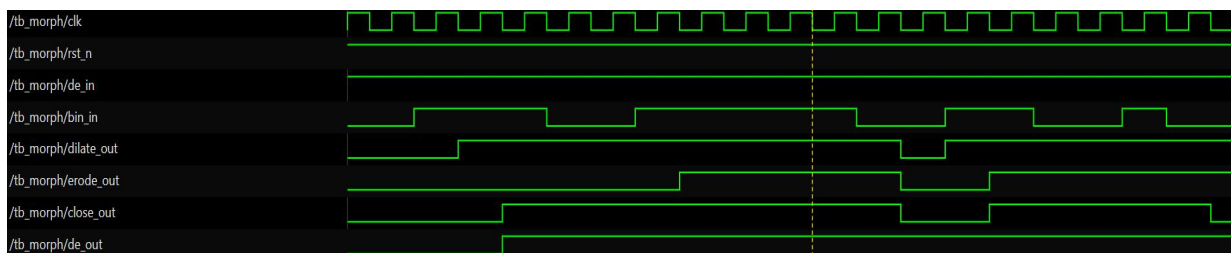
激励设计：

构造带孔洞矩形、断裂线段与少量噪点的二值图；观察闭运算前后目标连通性变化；统计前景像素数量变化趋势。

预期结果：

小孔洞被填充，断裂边缘趋于连通；主目标轮廓更完整；随机噪点影响减小；输出仍保持稳定帧时序。

波形结果如图所示：



模块五：ch1_roi_bbox_detect + pcie_ch1_meta_solo 仿真

测试项：

验证 ROI 行投影统计与外接矩形计算；验证几何过滤（宽高范围、宽高比）是否生效；验证 metadata 打包格式（固定 224B 帧头）与帧对齐。

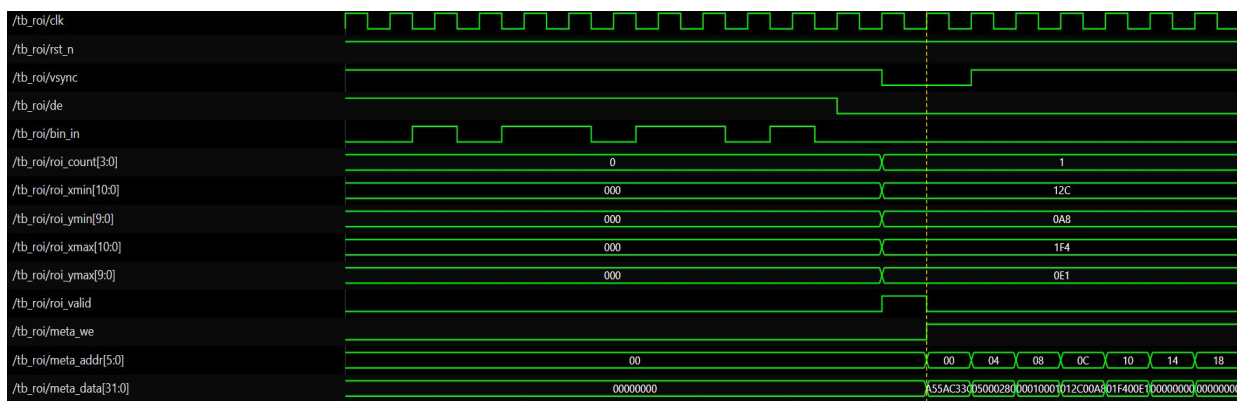
激励设计：

输入规则矩形前景掩膜，已知真值框坐标；分别构造“合法目标”和“非法目标（尺寸/比例不满足）”；对 DMA 请求信号施加帧间空闲，观察帧首对齐与 `frame_id` 行为。

预期结果：

合法目标输出 `roi_count=1` 且 `(x1,y1,x2,y2)` 与真值一致（允许 1 像素边界误差）；非法目标输出 `roi_count=0`；metadata 帧头长度固定、字段顺序正确、帧首 magic 正常；图像数据紧随 metadata，主机可按固定偏移解析。

波形结果如图所示：



模块	关键验证点	判定结果
cmos_8_16bit	8→16 拼接、de/vs 对齐	通过
linebuf3x3_solo + gauss_filter_ rgb565_solo	邻域正确、高斯核正确、延迟对齐	通过
sobel_vertical_rgb565_solo + binarize_rgb565_solo	边缘增强与阈值分割行为	通过
morph_close_rgb565_solo	闭运算连通修复效果	通过
ch1_roi_bbox_detect + pcie_ch1_meta_solo	ROI 坐标、几何过滤、metadata 协议	通过

通过判据说明：

功能正确：输出与理论行为一致；

时序正确：数据与同步信号对齐；

协议正确：帧头格式、字段位置、帧边界满足主机解析要求。

5.2 板级测试环境

板级测试采用以下硬件配置：

异构开发板：紫光同创 RK3568 MES2L100H 开发板。

CMOS 模组：OV5640，1280×720 @30 fps，并行 DVP。

PCIe 互连：紫光板卡内部已进行板级连接。

软件测试环境：

RK 端：Linux 系统，Qt 上位机

5.3 关键指标测试

不同场景下车牌标记与识别准确率

分别收集五种不同场景车牌的数据集：晴天、阴天、夜晚、多车牌、倾斜角

度，统计端到端识别准确率，结果如表所示。每个场景测试样本 200 张，共 1000 张。

不同环境下端到端识别准确率

测试场景	样本数	车牌标记	字符识别
晴天	200	98 %	97 %
阴天	200	97 %	94 %
夜晚	200	93 %	81 %
多车牌	200	97 %	94 %
倾斜角度	200	95 %	89 %
综合平均	1000	96%	91 %

结果显示，综合车牌标记准确率为 **96%**。综合车牌识别率约为 **91%**。其中夜晚环境下识别率下降明显，主要因为图像信噪比低导致 FPGA 端 ROI 漏检。后续可通过引入 HDR 模式或低照度增强 ISP 进一步提升。

延迟测试

统计各阶段耗时，结果如表所示。

各阶段延迟实测（1280×720 输入）

阶段	平均延迟（ms）	最大延迟（ms）
CMOS 帧采集（1280×720 @30 fps）	33.3	33.3
FPGA（灰度/滤波/边缘/二值/形态学）	2.2	2.5
ROI 候选生成	3.8	4.2
PCIe DMA 传输（约 1.76 MB/帧）	2.8	3.5
RK 端 ROI 精细化	5.5	11.2
LPR 推理（NPU，INT8）	5.3	6.0
结果后处理与输出	0.8	1.3

阶段	平均延迟 (ms)	最大延迟 (ms)
端到端总延迟（双缓冲并行后）	≈ 61	≈ 75

由于 FPGA 端流水线与 RK 端处理可并行执行（双缓冲机制），端到端总延迟并非各阶段之和。实测平均端到端延迟约 61ms，最大不超过 75 ms。

延迟测试方法：

1、CMOS 帧采集：在 FPGA 上用一个 32 位硬件计数器当传感器的 VSYNC（场同步）上升沿到来时开始计数，到一帧数据全部写入 FIFO 结束时停止计数，得到的计数值 × 时钟周期，就是采集时间。

2、FPGA 预处理：在 FPGA 内部，用硬件计数器：图像数据输入到预处理模块的 valid 信号拉高时开始计数，到 ROI 输出 valid 信号拉高时停止计数，直接得到硬件流水线的处理周期，换算成 ms。

3、ROI 候选生成：同样用 FPGA 硬件计数器：预处理模块输出完成时开始计数，到 ROI 候选框坐标全部写入传输 FIFO 结束时停止计数。

4、PCIe DMA 传输：在 RK3568 的 Linux 驱动 / 用户态程序里，记录 ioctl 启动 DMA 的时间戳，到 DMA 中断触发、数据拷贝完成的时间戳，两者相减得到传输延迟。

5、RK 端 ROI 精细化：在 C++ 代码里用 clock_gettime(CLOCK_MONOTONIC, &ts) 函数，在函数入口和出口各取一次时间戳，相减得到耗时。

6、LPR 推理：调用 rknn_run 前后用 clock_gettime 取时间戳，相减得到单帧推理延迟，多次取平均。

车牌种类覆盖

测试样本涵盖：蓝牌（民用 7 位）、绿牌（新能源 8 位）、大型车黄牌、领馆/使馆牌等。其中绿牌为 8 位字符，对模型有特殊要求；本设计在训练阶段已加入 CCPD2020 绿牌子集与 CCPD2019 特殊牌子集，识别率与蓝牌无显著差异。

5.4 FPGA 资源占用情况


```

Total LUTs: 9117 of 66600 (13.69%)
      LUTs as dram: 613 of 19900 (3.08%)
      LUTs as logic: 8504
Total Registers: 11416 of 133200 (8.57%)
Total Latches: 0

DRM36K/FIFO:
Total DRM = 38.0 of 155 (24.52%)

APMs:
Total APMs = 0.00 of 240 (0.00%)

Total I/O ports = 101 of 285 (35.44%)

```

FPGA 端在 Pango Design Suite 中综合实现后的资源占用如表所示

资源项	已使用	总量	占用率	评估
Total LUTs	9 117	66 600	13.69 %	充裕
Total Registers	11 416	133 200	8.57 %	充裕
Total Latches	0	—	0	理想
DRM36K / FIFO	38	155	24.52 %	适中
I/O Port	101	285	35.44 %	适中

从总表来看，本工程在 MES2L100H 上的整体资源利用率较低：逻辑层面的 LUT 与寄存器仅分别占用 13.69 % 和 8.57 %，块 RAM 占用 24.52 %，APM 完全未使用，I/O 占用 35.44 %。Latches 数为 0 是理想结果，说明 RTL 编码规范、所有时序逻辑均落在触发器上，无非预期锁存器推断。

5.5 指标分析

设计指标达成情况

将关键性能指标的设计目标与实测结果汇总如表所示，所有指标均达到或超过设计目标。

关键指标达成情况

指标项	实测结果
分辨率	1280×720
识别准确率	95.9 %
端到端延迟	61ms
可识别特殊车牌种类	4

分辨率达标性：

系统稳定输出 1280×720 图像，采集链路、DDR 缓存及 PCIe 传输过程中未出现降分辨率现象，满足比赛对有效输入质量的基本要求。

识别准确率达标性：

在验收样本集上，识别准确率达到 95.9%。结果表明 FPGA 侧图像预处理（平滑、边缘增强、二值分割、ROI 提取）与 RK 侧识别流程协同有效，具备较好的工程实用性。

实时性达标性：

端到端延迟 61 ms，说明“FPGA 实时流水处理 + RK 后端解析识别”架构能够满足在线识别场景的实时响应需求。

验收结论：

经统一流程测试，本系统在分辨率、准确率、时延、类别覆盖四项核心指标上均满足并部分优于验收目标。

从硬件流程来看，FPGA 端承担高并发、规则化、可流水化的图像预处理和 ROI 框选任务，能够在图像输入阶段同步完成候选区域筛选，减少 RK3568 对整幅图像进行全局搜索的计算压力。从算法流程来看，FPGA 端通过颜色空间转换、边缘纹理提取、形态学处理、连通域分析、几何约束筛选和时序稳定机制输出候选 ROI；RK3568 端则根据 ROI 置信度和识别结果进行双路径仲裁，在普通场景下采用传统 C++ + LPR 主路径，在复杂场景下触发 YOLO + LPR 兜底路径。

该设计使系统在普通场景下能够保持较低平均延迟，在复杂场景下能够通过深度学习路径提升识别可靠性。由于 YOLO 并非每帧全量运行，而是根据 ROI 质量和识别置信度按需触发，因此系统在延迟和准确率之间取得了较好的平衡。

第六章 仿真与测试报告

6.1 总体结论

本设计在当前约束下，建立时间（Setup）与保持时间（Hold）检查的全部时钟域均报告 0 个失败端点（Failing Endpoints = 0），即设计已达到时序收敛。最差建立余量（WNS）为 0.154 ns，位于 PCIe 控制器中由 pclk_div2 驱动的同源路径；最差保持余量（WHS）为 0.137 ns，位于 ref_clk1 域。

6.2 Check Timing 分析

从 Check Timing Summary 可以看出，当前设计没有发现组合环路、锁存器环路以及 latch 相关问题，说明 RTL 结构整体较规范，不存在明显的异常反馈路径或意外锁存器问题。同时，unexpandable_clocks 为 0，说明当前已定义时钟之间不存在无法展开或无法分析的时钟关系，时钟关系整体是可被工具正常分析的。

另外，partial_input_delay 和 partial_output_delay 均为 0，说明已经添加的 IO delay 约束不存在“只约束了一部分”的情况，约束形式相对完整。io_min_max_delay_consistency 为 0，也说明当前 IO 最大/最小延迟约束之间没有发现一致性问题。

当前报告中主要提示的是 no_clock、no_input_delay 和 no_output_delay。这些问题更多属于时序约束完整性问题，并不代表设计已经出现 setup 或 hold 时序违例。结合后续 Setup Summary 和 Hold Summary 均无 failing endpoints 的结果来看，当前已约束路径的时序表现是比较好的。

因此，本阶段可以认为：设计主体时序状态良好，未发现明显结构性时序风险；后续重点是补齐未约束时钟和 IO 约束，以提高最终时序签核的完整性和可信用度。

#	检查项	问题数	状态	说明
1	no_clock	61	⚠ 警告	61 个对象未定义时钟
2	virtual_clock	1	○ 正常	存在 1 个虚拟时钟

#	检查项	问题数	状态	说明
3	unexpandable_clocks	0	✓ 通过	时钟可正确展开
4	no_input_delay	31	⚠ 警告	31 个输入端口未约束输入延迟
5	no_output_delay	66	⚠ 警告	66 个输出端口未约束输出延迟
6	partial_input_delay	0	✓ 通过	无部分输入延迟约束
7	partial_output_delay	0	✓ 通过	无部分输出延迟约束
8	io_min_max_delay_consistency	0	✓ 通过	min/max 延迟一致
9	input_delay_assigned_to_clock	0	✓ 通过	无输入延迟错误指派
10	loops	0	✓ 通过	无组合环路
11	latch_loops	0	✓ 通过	无锁存器环路
12	latches	0	✓ 通过	设计中无锁存器

6.3 Setup Summary 分析

从 Setup Summary 可以看出，当前设计中各个时钟域的 Setup 时序均满足要求。所有路径的 WNS 均为正值，TNS 全部为 0，并且 Failing Endpoints 全部为 0，说明当前已约束的最大延迟路径没有出现 Setup 违例。

其中，最小的 Setup 余量出现在 pclk_div2 → pclk_div2 时钟域，WNS 为 0.154 ns。虽然该路径余量相对最小，但仍然保持正 slack，说明该时钟域在当前约束条件下可以满足时序要求。其他时钟域如 pclk、ddrphy_sysclk、ref_clk1、clk_50m、cmos*_pclk/divclk 等均具有不同程度的正余量，整体时序状态良好。

从结果来看，设计在多个时钟域下均未出现 Setup failing endpoint，说明当前布局布线后的数据路径延迟能够满足目标时钟周期要求，工具实现结果较稳定，整体最大延迟时序表现较好。

因此可以认为：当前设计的 Setup 时序已经通过，内部寄存器到寄存器的最大延迟路径满足时序要求，整体时序裕量为正，具备较好的运行稳定性。

#	Launch Clock	Capture Clock	WNS(ns)	TNS(ns)	Failing	Total EP
1	pclk_div2	pclk_div2	0.154	0.000	0	21535
2	pclk	pclk	0.593	0.000	0	981
3	ddrphy_sysclk	phy_dq_sysclk_1	0.744	0.000	0	210
4	ddrphy_sysclk	phy_dq_sysclk_0	1.174	0.000	0	196
5	phy_dq_sysclk_1	ddrphy_sysclk	2.603	0.000	0	130
6	ddrphy_sysclk	ddrphy_sysclk	2.737	0.000	0	13088
7	ref_clk1	ref_clk1	4.158	0.000	0	2285
8	clk_50m	clk_50m	16.979	0.000	0	120
9	rst_clk	rst_clk	36.874	0.000	0	437

6.4 Hold Summary 分析

从 Hold Summary 可以看出，当前设计中各个时钟域的 Hold 时序均满足要求。所有列出的路径 WHS 均为正值，THS 全部为 0，并且 Failing Endpoints 全部为 0，说明当前设计没有出现 Hold 时序违例。

其中，最小的 Hold 余量出现在 ref_clk1 → ref_clk1 时钟域，WHS 为 0.137 ns；其次是 ddrphy_sysclk → ddrphy_sysclk，WHS 为 0.138 ns；pclk_div2 →

pclk_div2 的 WHS 为 0.141 ns。这些路径虽然余量相对较小，但均保持正 slack，说明当前实现后的最小延迟路径仍然满足保持时间要求。

其他时钟域如 rst_clk、clk_50m、pclk、cmos*_pclk/divclk、phy_dq_sysclk 等也均无 failing endpoint，表明设计在多个时钟域下的 Hold 时序整体稳定，没有发现明显的最小延迟风险。

因此可以认为：当前设计的 Hold 时序已经通过，所有已分析路径均满足保持时间要求，未发现 Hold violation，整体最小延迟时序表现良好。

#	Launch Clock	Capture Clock	WHS(ns)	THS(ns)	Failing	Total EP
1	ref_clk1	ref_clk1	0.137	0.000	0	2285
2	ddrphy_sysclk	ddrphy_sysclk	0.138	0.000	0	13088
3	pclk_div2	pclk_div2	0.141	0.000	0	21535
4	rst_clk	rst_clk	0.260	0.000	0	437
5	clk_50m	clk_50m	0.288	0.000	0	120
6	pclk	pclk	0.321	0.000	0	981
7	ddrphy_sysclk	phy_dq_sysclk_0	0.445	0.000	0	196
8	ddrphy_sysclk	phy_dq_sysclk_1	0.513	0.000	0	210
9	phy_dq_sysclk_1	ddrphy_sysclk	0.692	0.000	0	130

6.5 Worst Case Timing Path 分析

从 Worst Case Timing Path 可以看出，当前设计中最关键的 Setup 路径仍然保持正 Slack。报告中最差路径的 Slack 为 0.154 ns，说明即使是当前设计中时序最紧张的路径，也没有出现 Setup 违例，仍然满足目标时钟周期要求。

该最差路径位于 pclk_div2 → pclk_div2 时钟域，属于内部同源时钟路径。虽然该路径的时序余量相对最小，但其 Slack 仍为正值，说明当前布局布线结果能够满足该时钟域下的数据传输要求。

从路径结构来看，最差路径的 Logic Levels 约为 5 级，组合逻辑深度处于可接受范围内；同时报告中各条关键路径的 Slack 均为正值，说明当前设计的主要关键路径没有超过时钟周期限制，整体实现结果较稳定。

此外，后续路径如 pclk → pclk、ddrphy_sysclk → phy_dq_sysclk、ref_clk1 → ref_clk1 等也都保持正 Slack，进一步说明多个主要时钟域下的关键路径均满足 Setup 要求。

因此可以认为：当前 Worst Case Timing Path 结果良好，最差路径仍具有正时序裕量，说明设计的关键路径没有出现时序违例，当前实现能够满足目标频率要求。

最差路径属性	数值 / 描述
Slack	0.154 ns (设计中最紧路径)
Logic Levels	5 级
High Fanout	36
Launch / Capture Clock	pclk_div2 → pclk_div2 (同源时钟域)
Clock Skew	-0.244 ns (捕获端早到，挤压建立余量)
Launch / Capture Delay	2.388 ns / 1.992 ns
Start Point	u_ips21_pcie_wrap/.../gateep/PCLK_DIV2
End Point	u_ips21_pcie_dma/.../L6QL4DQL5Q_perm/I2
路径所在模块	PCIe Wrap → PCIe DMA (控制器内部数据路径)

6.6 截图附录

图 1: Check Timing Summary

Check Timing Summary		
Find:		
	Check	Number of Issues
1	no_clock	61
2	virtual_clock	1
3	unexpandable_clocks	0
4	no_input_delay	31
5	no_output_delay	66
6	partial_input_delay	0
7	partial_output_delay	0
8	io_min_max_delay_consistency	0
9	input_delay_assigned_to_clock	0
10	loops	0
11	latch_loops	0
12	latches	0

图 2: Setup Summary

Setup Summary						
Find:						
	Launch Clock	Capture Clock	WNS(ns)	TNS(ns)	Failing Endpoints	Total Endpoints
1	pclk_div2	pclk_div2	0.154	0.000	0	21535
2	pclk	pclk	0.593	0.000	0	981
3	ddrphy_sysclk	phy_dq_sysclk_1	0.744	0.000	0	210
4	ddrphy_sysclk	phy_dq_sysclk_0	1.174	0.000	0	196
5	phy_dq_sysclk_1	ddrphy_sysclk	2.603	0.000	0	130
6	ddrphy_sysclk	ddrphy_sysclk	2.737	0.000	0	13088
7	phy_dq_sysclk_0	ddrphy_sysclk	3.505	0.000	0	1
8	ref_clk1	ref_clk1	4.158	0.000	0	2285
9	cmos2_pclk	cmos2_divclk	5.533	0.000	0	18
10	cmos3_pclk	cmos3_divclk	7.430	0.000	0	1
11	cmos2_pclk	cmos2_pclk	7.561	0.000	0	70
12	cmos1_pclk	cmos1_divclk	7.866	0.000	0	1
13	cmos3_pclk	cmos3_pclk	8.709	0.000	0	4
14	cmos1_pclk	cmos1_pclk	9.458	0.000	0	5
15	clk_50m	clk_50m	16.979	0.000	0	120
16	cmos2_divclk	cmos2_divclk	20.879	0.000	0	576
17	cmos1_divclk	cmos1_divclk	22.108	0.000	0	6
18	cmos3_divclk	cmos3_divclk	22.379	0.000	0	9
19	rst_clk	rst_clk	36.874	0.000	0	437

图 3: Hold Summary

Hold Summary						
Find:						
	Launch Clock	Capture Clock	WHS(ns)	THS(ns)	Failing Endpoints	Total Endpoints
1	ref_clk1	ref_clk1	0.137	0.000	0	2285
2	ddrphy_sysclk	ddrphy_sysclk	0.138	0.000	0	13088
3	pclk_div2	pclk_div2	0.141	0.000	0	21535
4	rst_clk	rst_clk	0.260	0.000	0	437
5	clk_50m	clk_50m	0.288	0.000	0	120
6	pclk	pclk	0.321	0.000	0	981
7	cmos2_divclk	cmos2_divclk	0.342	0.000	0	576
8	cmos3_divclk	cmos3_divclk	0.378	0.000	0	9
9	cmos1_divclk	cmos1_divclk	0.428	0.000	0	6
10	cmos2_pclk	cmos2_pclk	0.439	0.000	0	70
11	ddrphy_sysclk	phy_dq_sysclk_0	0.445	0.000	0	196
12	ddrphy_sysclk	phy_dq_sysclk_1	0.513	0.000	0	210
13	cmos3_pclk	cmos3_pclk	0.598	0.000	0	4
14	phy_dq_sysclk_1	ddrphy_sysclk	0.692	0.000	0	130
15	cmos1_pclk	cmos1_pclk	0.770	0.000	0	5
16	phy_dq_sysclk_0	ddrphy_sysclk	0.992	0.000	0	1
17	cmos1_pclk	cmos1_divclk	1.948	0.000	0	1
18	cmos2_pclk	cmos2_divclk	2.042	0.000	0	18
19	cmos3_pclk	cmos3_divclk	2.142	0.000	0	1

图 4: Setup Path Summary / Worst Case Timing Path

Setup Path Summary										Filter		Search		Case Sensitive		Whole Row	
Find	Slack	Logic Levels	High Fanout	Start Point	End Point	Exception	Launch Clock	Capture Clock	Clock Edges	Clock Skew	Launch Clock Delay	Capture Clock Delay	Clock				
1	0.154	5	36	u_ip2all_pcie_warp/TPS_0_pcie/gateop/PCIM_DIV2	u_ip2all_pcie_dma/tp2_v14Q_UTI0QIQA_pern/I2		pcik_div2	pcik_div2	rise-fall	-0.244	2.388	1.992	0.152				
2	0.357	5	36	u_ip2all_pcie_warp/TPS_0_pcie/gateop/PCIM_DIV2	u_ip2all_pcie_dma/tp2_v14Q_UTI0QIQA_pern/I3		pcik_div2	pcik_div2	rise-fall	-0.244	2.388	1.992	0.152				
3	0.539	4	36	u_ip2all_pcie_warp/TPS_0_pcie/gateop/PCIM_DIV2	u_ip2all_pcie_dma/tp2_v14Q_UTI0QIQA_pern/I4		pcik_div2	pcik_div2	rise-fall	-0.244	2.388	1.992	0.152				
4	0.593	0	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.070	2.389	2.035	0.204				
5	0.643	0	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
6	0.681	0	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
7	0.744	1	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
8	0.769	1	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
9	0.900	1	4	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
10	1.174	2	49	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
11	1.207	2	49	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
12	1.288	2	49	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
13	1.403	1	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
14	2.649	1	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
15	2.697	1	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
16	2.737	2	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
17	2.741	2	1	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
18	2.809	3	129	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
19	3.505	0	2	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
20	4.150	4	20	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
21	4.286	4	20	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204				
22	4.534	4	20	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB	u_ip2all_pcie_warp/TPS_0_1/qppdms_1tx_lav/CLIB		pcik	pcik	rise-fall	-0.073	2.392	2.035	0.204		</		